GraphStream-IDS

Continual Graph Neural Network for Zero-Day Intrusion Detection Under Network Concept Drift

Project type: Advanced AI/ML research project **Domain:** Cybersecurity **Publication goal:** Research paper

Executive overview

GraphStream-IDS is a research-oriented network intrusion detection system that represents communication as a dynamic graph and continuously learns as network behavior changes. Its objective is to detect known attacks, identify previously unseen or zero-day attack behavior, and retain knowledge of earlier attacks without retraining the entire model from scratch.

> The project is designed as a defensive research system. It should be evaluated using public datasets and controlled simulations rather than real-world attacks or unauthorized network activity.

1. Motivation and research gap

Static intrusion-detection models can lose effectiveness when traffic patterns and attack techniques change. This is called concept drift. Continual-learning research in intrusion detection focuses on learning new patterns while avoiding catastrophic forgetting of previously learned classes. [1]

A further limitation is that ordinary tabular classifiers do not explicitly model relationships between hosts, services, ports, and communication sessions. A graph representation can capture these relationships and provide a foundation for detecting unusual communication structures.

> **Research gap:** Develop and evaluate a memory-efficient dynamic GNN that combines drift detection, selective replay, and uncertainty estimation for zero-day intrusion detection.

2. Main research question

Can a dynamic graph neural network with concept-drift detection and selective memory replay detect unseen cyberattacks more effectively than static machine-learning models and conventional continual-learning methods?

Proposed hypotheses

- Graph-based representations capture network communication relationships better than ordinary tabular classifiers.
- Continual learning improves performance when traffic and attack distributions change over time.
- Selective replay reduces catastrophic forgetting while using less memory than storing the complete historical dataset.
- Uncertainty estimation helps the system flag unknown behavior instead of forcing it into an incorrect known class.

3. Proposed architecture

Component	Function
Network flow records	Packet duration, protocol, source/destination information, bytes, flags, and connection statistics.
Dynamic graph construction	Nodes represent hosts, IPs, ports, or services; edges represent communication events and contain flow features.
Graph encoder	GraphSAGE, GAT, or a temporal GNN learns node, edge, and graph-level patterns.
Known-attack classifier	Predicts normal traffic and attack categories observed during training.
Unknown-attack detector	Uses distance, energy, density, or uncertainty scores to flag unfamiliar behavior.
Drift detector	Monitors changes in feature or embedding distributions over time.
Selective replay memory	Stores representative historical samples to reduce catastrophic forgetting.
Alert output	Returns predicted class, confidence, novelty score, and supporting traffic features.

Recommended initial model: GraphSAGE or GAT combined with a lightweight replay buffer. Add a temporal GNN only after the static graph pipeline is working reliably.

4. Datasets and data protocol

Recommended public datasets include CIC-IDS2017, CSE-CIC-IDS2018, UNSW-NB15, and TON-IoT. The Canadian Institute for Cybersecurity provides public intrusion-detection datasets for research. [2]

Use chronological evaluation rather than only random train-test splitting. Train on earlier traffic, introduce new traffic periods sequentially, and reserve at least one attack family as an unseen or zero-day test condition. Dataset leakage, duplicate flows, class imbalance, and unrealistic splits must be checked because they can produce misleadingly high results. [3]

Suggested setup

1. Phase 1: Train on normal traffic and selected known attack families.
2. Phase 2: Stream later traffic batches to simulate changing network conditions.
3. Phase 3: Introduce an unseen attack family and evaluate novelty detection.
4. Phase 4: Test replay memory, drift detection, and recovery from distribution change.

5. Baselines and evaluation

Model	Purpose
Random Forest / XGBoost	Classical tabular baseline
MLP or 1D-CNN	Deep tabular baseline
LSTM	Sequential traffic baseline
Static GNN	Tests graph representation
GNN + ordinary replay	Tests continual learning
GraphStream-IDS	Complete proposed method

Report macro-F1, precision, recall, false-positive rate, zero-day detection rate, AUROC/AUPRC, time to detection, catastrophic-forgetting score, memory usage, and inference latency. Include confidence intervals across multiple

random seeds.

6. Publication contribution

The paper should claim one focused contribution rather than presenting a collection of disconnected features:

> **Proposed contribution:** A dynamic, memory-efficient GNN framework that combines drift-aware learning, selective replay, and uncertainty-based unknown-attack detection under realistic chronological and non-IID traffic conditions.

Required ablation studies

- Without graph structure.
- Without temporal information.
- Without drift detection.
- Random replay instead of selective replay.
- Without uncertainty estimation.
- Different replay-buffer sizes and drift thresholds.

7. Technology stack

Python, PyTorch, PyTorch Geometric or DGL, scikit-learn, XGBoost, River for online learning and drift detection, NetworkX, MLflow or Weights & Biases, and Streamlit for visualization. Begin with a small CNN or GraphSAGE model on Google Colab before attempting larger temporal models.

8. Six-month roadmap

Weeks	Deliverable
1–3	Literature review, gap definition, dataset audit
4–7	Preprocessing pipeline and classical baselines
8–11	Static graph construction and GNN baseline
12–16	Continual-learning and replay implementation
17–20	Drift detection and zero-day protocol
21–23	Uncertainty module and ablations
24–26	Results, paper writing, code cleanup, venue selection

9. Paper outline

1. Introduction and motivation
2. Related work on GNNs, intrusion detection, and continual learning
3. Threat model and problem definition
4. GraphStream-IDS methodology
5. Datasets and chronological evaluation protocol
6. Results and ablation studies
7. Error analysis, limitations, and ethical considerations
8. Conclusion and reproducibility statement

Research caution

> This is a defensive research project. Work only with public datasets, isolated lab environments, and permission-based traffic captures. Do not scan, attack, or test systems that you do not own or have explicit authorization to assess.

Selected references

- [1] Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection, arXiv:2412.16264.
- [2] Canadian Institute for Cybersecurity, public datasets and research resources.
- [3] Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes.